

4.4 Class Diagram

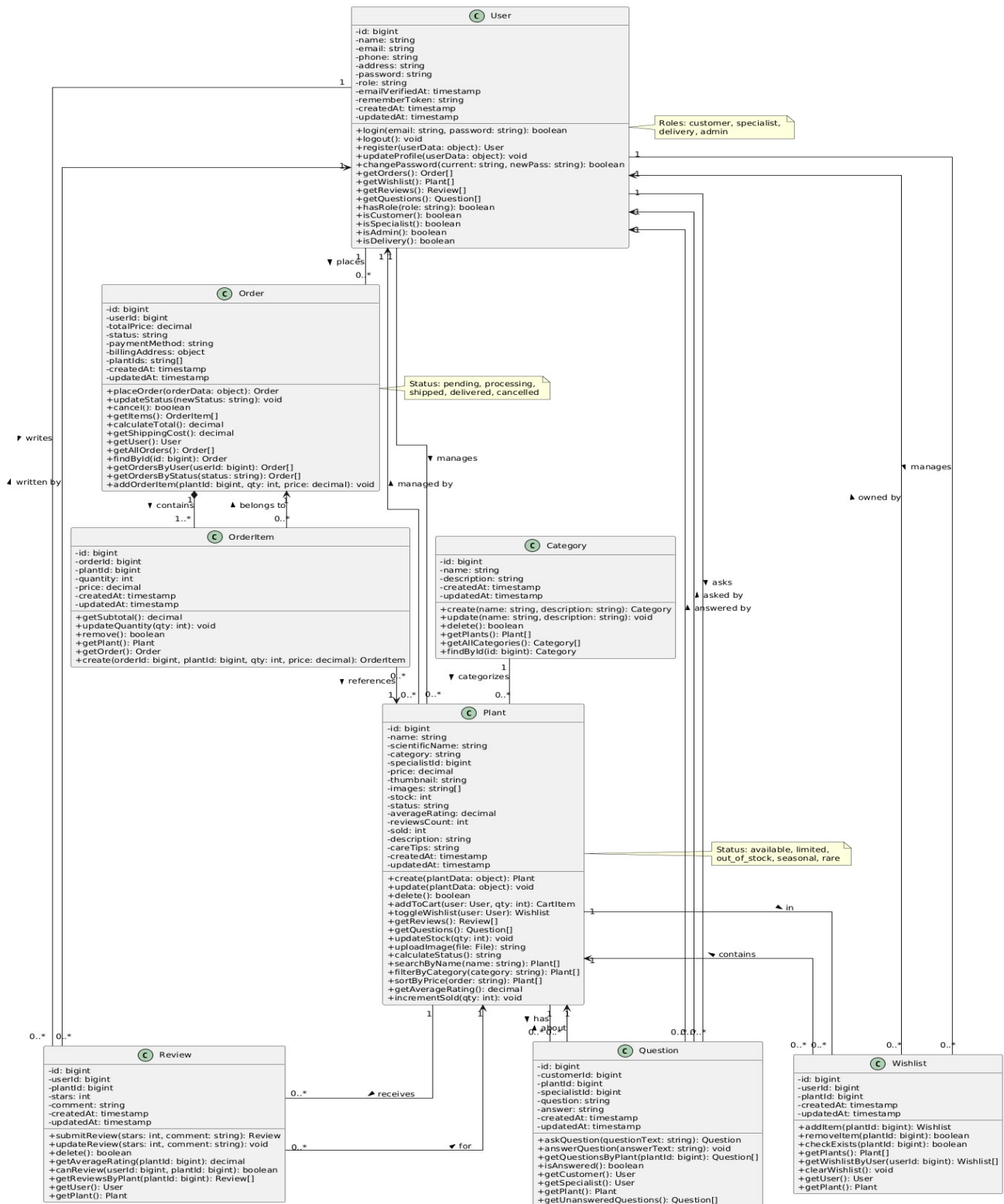


Figure 4.15: Class Diagram of the Plantify System

Chapter 5.
Project Management

5.1 Risk Identification Table

Table 5.1: Risk Identification:

Risk ID	Risk Category	Risk Description	Potential Impact
R-01	Technology	Database connection failure or misconfiguration (MySQL .env errors)	Application cannot serve any data; complete system downtime
R-02	Technology	Missing token-based authentication middleware; all API routes unprotected	Unauthorised access to any endpoint; data leakage and manipulation
R-03	Technology	No role-based access control (RBAC) enforcement on API endpoints	Privilege escalation; customers can perform admin/specialist operations
R-04	Technology	Frontend build failure due to TypeScript strict mode and compilation errors	Unable to deploy frontend; development blocked
R-05	Technology	Cart state loss on browser refresh or navigation away	Customers lose selected items; increased cart abandonment
R-06	Estimation	Inadequate test coverage – no formal unit or integration tests exist	Undetected regressions reach production; reliability unverified
R-07	Technology	CORS misconfiguration blocking frontend-backend communication	Complete API call failure; all frontend features broken

Risk ID	Risk Category	Risk Description	Potential Impact
R-08	Technology	File upload failure (storage symlink missing, MIME type, or size issues)	Plant images cannot be uploaded or displayed

5.2 Risk Analysis Table

Table 5.2: Risk Analysis:

Risk ID	Probability	Effects	Risk Severity	Rationale
R-01	Moderate	Serious	High	Local .env misconfiguration is common but results in a complete outage.
R-02	High	Catastrophic	Critical	API routes currently have no auth middleware; widespread exposure of sensitive data.
R-03	High	Catastrophic	Critical	No middleware-based role checks exist; privilege escalation is trivial.
R-04	Moderate	Serious	High	TypeScript strict mode frequently surfaces type mismatches; halts deployment.
R-05	Moderate	Serious	High	Only localStorage-based cart; no server sync; refresh discards state, risking lost sales.

Risk ID	Probability	Effects	Risk Severity	Rationale
R-06	High	Serious	High	No automated test suite; manual verification only; high risk of regressions.
R-07	Low	Serious	Medium	CORS config exists but port mismatches occur; if triggered, total API failure.
R-08	Moderate	Tolerable	Medium	Upload validation present but symlink may be missing; partial image failure.

5.3 Risk Planning Table

Table 5.3: Risk Planning:

Risk ID	Severity	Response Strategy	Planned Action
R-01	High	Mitigate	Create a .env.example with clear setup instructions; add a database health-check endpoint.
R-02	Critical	Mitigate	Install Laravel Sanctum; generate API tokens on login; apply auth:sanctum middleware to all protected routes.
R-03	Critical	Avoid / Mitigate	Implement role-based middleware (IsAdmin, IsSpecialist, IsDelivery); enforce on every restricted route.

Risk ID	Severity	Response Strategy	Planned Action
R-04	High	Mitigate	Enforce pre-commit TypeScript linting; run npm run build in CI; fix all type errors early.
R-05	High	Mitigate	Implement localStorage cart persistence with recovery on page load; optionally sync cart with backend on login.
R-06	High	Mitigate	Write PHPUnit tests for Auth, Plant, and Order controllers; add basic frontend component tests.
R-07	Medium	Mitigate	Standardise API base URL as a single config variable; verify CORS permits both localhost:8000 and 8001.
R-08	Medium	Mitigate	Verify storage:link existence on every deployment; add client-side file type/size validation; document upload limits.

5.4 The RMMM Plan

R-01: Database Connection Failure

Table 5.4: RMMM Plan – R-01 Database Connection Failure:

Risk Information Sheet	Information
Risk ID	R-01
Date	(project start)
Probability	Moderate
Impact	Serious
Description	MySQL connection failure due to incorrect .env configuration or missing database service. Application is unable to serve data.
Refinement / Context	Subcondition 1 – .env file missing or wrong credentials. Subcondition 2 – MySQL service not running.
Mitigation / Monitoring	1. Provide .env.example with placeholder values. 2. Add a /api/health endpoint that checks DB connectivity. 3. Document setup steps in README. 4. Weekly manual health-check during sprints.
Management / Contingency Plan / Trigger	If the health endpoint returns 500 or a connection error occurs, restore the latest SQL dump and reconfigure .env from the template. Trigger: health check fails at any time.
Current Status	.env.example created; health endpoint to be implemented in Sprint 1.

Risk Information Sheet	Information
Originator	Student Developer
Assigned	Student Developer

R-02: Missing Authentication Middleware

Table 5.5: RMMM Plan – R-02 Missing Authentication Middleware:

Risk Information Sheet	Information
Risk ID	R-02
Date	(project start)
Probability	High
Impact	Catastrophic
Description	API routes lack token-based authentication. Any request, including unauthenticated ones, can access protected endpoints.
Refinement / Context	Subcondition 1 – routes/api.php currently defines no auth middleware. Subcondition 2 – Sanctum package not yet installed.
Mitigation / Monitoring	1. Install Laravel Sanctum (composer require laravel/sanctum). 2. Add auth:sanctum middleware to all non-public routes. 3. Weekly route audit via php artisan route:list to confirm coverage.

Risk Information Sheet	Information
Management / Contingency Plan / Trigger	If any unprotected endpoint is discovered, immediately apply auth middleware and invalidate exposed tokens. Trigger: unprotected route found during audit.
Current Status	Sanctum installation and middleware application planned for Sprint 1.
Originator	Student Developer
Assigned	Student Developer

R-03: No Role-Based Access Control

Table 5.6: RMMM Plan – R-03 No Role-Based Access Control:

Risk Information Sheet	Information
Risk ID	R-03
Date	(project start)
Probability	High
Impact	Catastrophic
Description	API endpoints do not enforce role-based restrictions. A customer could call specialist or admin routes, leading to data corruption and security breaches.

Risk Information Sheet	Information
Refinement / Context	Subcondition 1 – User model has a 'role' column, but no middleware checks it. Subcondition 2 – Sensitive routes (user management, plant creation) are unprotected.
Mitigation / Monitoring	1. Create custom middleware: IsAdmin, IsSpecialist, IsDelivery. 2. Apply middleware to corresponding route groups. 3. Test with role-specific tokens each sprint.
Management / Contingency Plan / Trigger	If a non-admin successfully accesses an admin endpoint, patch middleware assignment within 24 hours and audit recent logs. Trigger: privilege escalation detected.
Current Status	Middleware classes to be created and applied in Sprint 1 alongside Sanctum.
Originator	Student Developer
Assigned	Student Developer

R-04: Frontend Build Failure

Table 5.7: RMMM Plan – R-04 Frontend Build Failure:

Risk Information Sheet	Information
Risk ID	R-04
Date	(Sprint 2 start)
Probability	Moderate

Risk Information Sheet	Information
Impact	Serious
Description	Next.js frontend fails to build because of TypeScript strict-mode type errors, blocking deployment.
Refinement / Context	Subcondition 1 – Type mismatches in API response handling. Subcondition 2 – Missing or incorrect prop types in React components.
Mitigation / Monitoring	1. Enforce pre-commit linting with ESLint and tsc --noEmit. 2. Add a CI step (GitHub Actions) that runs npm run build on every push. 3. Fix type errors as they appear.
Management / Contingency Plan / Trigger	If build fails on main branch, revert to the last known good commit and fix errors incrementally. Trigger: build failure in CI or local production build.
Current Status	Pre-commit hooks to be configured in Sprint 2; CI pipeline planned for Sprint 2.
Originator	Student Developer
Assigned	Student Developer

R-05: Cart State Loss

Table 5.8: RMMM Plan – R-05 Cart State Loss:

Risk Information Sheet	Information
Risk ID	R-05
Date	(Sprint 2 start)
Probability	Moderate
Impact	Serious
Description	Shopping cart data stored only in browser localStorage is lost on page refresh, browser crash, or navigation away, causing abandoned checkouts.
Refinement / Context	Subcondition 1 – No server-side cart persistence. Subcondition 2 – localStorage may be cleared by the user or browser policy.
Mitigation / Monitoring	1. Ensure cart state is read from localStorage on every page load. 2. Add a warning before leaving the checkout page. 3. (Medium term) Sync cart to backend on user login.
Management / Contingency Plan / Trigger	If the cart is found empty after a refresh, attempt to recover from the latest localStorage snapshot; if unavailable, display an empty cart message with a clear CTA. Trigger: cart items count drops to zero unexpectedly.
Current Status	localStorage persistence implemented; recovery logic to be refined in Sprint 2.
Originator	Student Developer

Risk Information Sheet	Information
Assigned	Student Developer

R-06: Inadequate Test Coverage

Table 5.9: RMMM Plan – R-06 Inadequate Test Coverage:

Risk Information Sheet	Information
Risk ID	R-06
Date	(Sprint 3 start)
Probability	High
Impact	Serious
Description	No automated unit or integration tests exist. All verification is manual, making regressions likely as the codebase evolves.
Refinement / Context	Subcondition 1 – Critical backend controllers (Auth, Plant, Order) lack PHPUnit tests. Subcondition 2 – Frontend components have no formal test suite.
Mitigation / Monitoring	1. Write PHPUnit tests for Auth, Plant, and Order controllers covering success and validation failures. 2. Add basic component smoke tests. 3. Run test suite weekly and require passing tests before merges.
Management / Contingency Plan / Trigger	If a previously passing test fails after a code change, fix the test or the code immediately and add a regression test. Trigger: any test failure in the CI run.

Risk Information Sheet	Information
Current Status	Test suite development scheduled for Sprint 3.
Originator	Student Developer
Assigned	Student Developer

R-07: CORS Misconfiguration

Table 5.10: RMMM Plan – R-07 CORS Misconfiguration:

Risk Information Sheet	Information
Risk ID	R-07
Date	(Sprint 1)
Probability	Low
Impact	Serious
Description	CORS settings in Laravel may not allow the frontend origin (port mismatch between :8000 and :8001), causing all API calls to fail.
Refinement / Context	Subcondition 1 – Laravel’s config/cors.php may only whitelist the production domain. Subcondition 2 – Frontend dev server (Next.js) often runs on port 3000 or 3001, while Laravel uses 8000.
Mitigation / Monitoring	1. Store API base URL in a single environment variable. 2. Add localhost:3000 and localhost:3001 to allowed origins. 3.

Risk Information Sheet	Information
	Test a cross-origin request from frontend to backend at the start of each sprint.
Management / Contingency Plan / Trigger	If CORS errors appear in the browser console, adjust cors.php to explicitly allow the frontend origin and re-verify with curl. Trigger: any CORS error during development.
Current Status	CORS configuration verified for current dev ports; will be re-checked after deployment.
Originator	Student Developer
Assigned	Student Developer

R-08: File Upload Failure

Table 5.11: RMMM Plan – R-08 File Upload Failure:

Risk Information Sheet	Information
Risk ID	R-08
Date	(Sprint 1)
Probability	Moderate
Impact	Tolerable
Description	Plant image uploads fail because the storage symbolic link is missing, or uploaded files exceed size/type limits, resulting in broken images.

Risk Information Sheet	Information
Refinement / Context	Subcondition 1 – php artisan storage:link not executed after deployment. Subcondition 2 – Client sends an image >10MB or an unsupported format without proper client-side validation.
Mitigation / Monitoring	1. Document storage:link command in setup instructions. 2. Add a startup script that checks symlink existence. 3. Implement client-side file size/type validation before upload. 4. Test the upload endpoint after every deployment.
Management / Contingency Plan / Trigger	If image upload returns 500 or image URL returns 404, re-run storage:link and ensure the uploads directory has correct permissions. Trigger: a plant's image fails to display.
Current Status	Backend validation in place; symlink check to be added to deployment steps.
Originator	Student Developer
Assigned	Student Developer

Chapter 6.
Project Planning

6.1 System Project Estimation

The Plantify project is a full-stack plant nursery management application built with Laravel 12 (PHP) and Next.js 16 (React/TypeScript). It supports four user roles – Customer, Specialist, Delivery, and Admin – across 8 database tables and 20+ API endpoints. As a single-developer, 13-week academic project, the estimation has been carried out using the IFPUG Function Point Analysis (FPA) method, combined with a productivity rate calibrated for modern web frameworks and a solo developer. The following sections present the detailed estimation, the resulting 13-week schedule, and the complete budget in Bangladeshi Taka (TK).

6.2 Function-Oriented Metrics

Function Point Analysis categorises software functionality into five standard function types:

Table 6.1: Function-Oriented Metrics (IFPUG Categories):

Type	Abbreviation	Description
External Input	EI	Processes data or control information entering the system from outside its boundary.
External Output	EO	Sends data or control information out of the system, after performing calculations or logic.
External Inquiry	EQ	Retrieves data from inside the system and presents it without altering it or performing computations.
Internal Logical File	ILF	A user-identifiable group of logically related data maintained within the application (database tables).
External Interface File	EIF	A user-identifiable group of logically related data referenced but not maintained by the application; read-only.

6.3 Identifying Complexity

6.3.1 Complexity Weights

The standard IFPUG complexity weights used in this estimation are:

Table 6.2: IFPUG Complexity Weights:

Function Type	Low	Average	High
External Input (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiry (EQ)	3	4	6
Internal Logical File (ILF)	7	10	15
External Interface File (EIF)	5	7	10

6.3.2 Identifying Complexity of Transaction Functions

All transaction functions (EI, EO, EQ) are listed with their Data Element Types (DETs), File Types Referenced (FTRs), assigned complexity, and corresponding weight.

External Inputs (EI) – 18 functions

Table 6.3: External Inputs (EI) Complexity Analysis:

EI ID	Function Description	DETs	FTRs	Complexity	Weight
EI-01	Register new user account	4	1	Low	3
EI-02	User login	2	1	Low	3
EI-03	Create plant listing	10	2	Average	4
EI-04	Update plant listing	10	2	Average	4
EI-05	Delete plant listing	1	1	Low	3

EI ID	Function Description	DETs	FTRs	Complexity	Weight
EI-06	Place new order	5	3	High	6
EI-07	Update order status	2	1	Low	3
EI-08	Submit plant review	4	2	Low	3
EI-09	Ask question about plant	3	2	Low	3
EI-10	Answer a customer question	2	1	Low	3
EI-11	Add plant to wishlist	2	1	Low	3
EI-12	Remove plant from wishlist	1	1	Low	3
EI-13	Create new category	3	1	Low	3
EI-14	Update category	3	1	Low	3
EI-15	Upload plant image file	1	1	Low	3
EI-16	Create user (admin)	5	1	Low	3
EI-17	Update user (admin)	5	1	Low	3
EI-18	Delete user (admin)	1	1	Low	3

External Outputs (EO) – 2 functions

Table 6.4: External Outputs (EO) Complexity Analysis:

EO ID	Function Description	DETs	FTRs	Complexity	Weight
EO-01	Order confirmation receipt	8	2	Average	5
EO-02	Admin analytics dashboard	12	4	High	7

External Inquiries (EQ) – 12 functions

Table 6.5: External Inquiries (EQ) Complexity Analysis:

EQ ID	Function Description	DETs	FTRs	Complexity	Weight
EQ-01	Browse / filter all plants	6	1	Low	3
EQ-02	View single plant details	10	3	Average	4
EQ-03	List user orders (customer)	5	2	Low	3
EQ-04	List all categories	3	1	Low	3
EQ-05	View reviews for a plant	3	1	Low	3
EQ-06	View questions for a plant	3	1	Low	3
EQ-07	View wishlist items	4	2	Low	3
EQ-08	Check wishlist status	1	1	Low	3
EQ-09	Admin – list all users	5	1	Low	3
EQ-10	Admin – list all orders	5	2	Low	3
EQ-11	Admin – view single user	5	1	Low	3
EQ-12	Delivery – view assigned orders	4	2	Average	4

6.3.3 Identifying Complexity of Data Functions

Internal Logical Files (ILF) – 8 files

Table 6.6: Internal Logical Files (ILF) Complexity Analysis:

ILF ID	Data Entity	RETs	DETs	Complexity	Weight
ILF-01	Users	1	8	Low	7
ILF-02	Plants	1	12	Low	7

ILF ID	Data Entity	RETs	DETs	Complexity	Weight
ILF-03	Categories	1	4	Low	7
ILF-04	Orders	1	7	Low	7
ILF-05	Order Items	1	5	Low	7
ILF-06	Reviews	1	6	Low	7
ILF-07	Questions	1	7	Low	7
ILF-08	Wishlists	1	4	Low	7

External Interface Files (EIF) – 1 file

Table 6.7: External Interface Files (EIF) Complexity Analysis:

EIF ID	External Data Source	RETs	DETs	Complexity	Weight
EIF-01	File Storage (public/uploads)	1	2	Low	5

6.3.4 Unadjusted Function Point Contribution (Transactions)

Table 6.8: Unadjusted Function Point Contribution – Transactions:

Function Type	Count	Low	Average	High	Total UFP
External Inputs (EI)	18	$15 \times 3 = 45$	$2 \times 4 = 8$	$1 \times 6 = 6$	59
External Outputs (EO)	2	0	$1 \times 5 = 5$	$1 \times 7 = 7$	12
External Inquiries (EQ)	12	$10 \times 3 = 30$	$2 \times 4 = 8$	0	38

Function Type	Count	Low	Average	High	Total UFP
Total UFP (Transactions)	32				109

6.3.5 Unadjusted Function Point Contribution (Data)

Table 6.9: Unadjusted Function Point Contribution – Data:

Function Type	Count	Low	Average	High	Total UFP
Internal Logical Files (ILF)	8	$8 \times 7 = 56$	0	0	56
External Interface Files (EIF)	1	$1 \times 5 = 5$	0	0	5
Total UFP (Data)	9				61

Unadjusted Function Points (UFP) = 109 + 61 = 170

6.3.6 Performance and Environmental Impact

The Value Adjustment Factor (VAF) is derived from the 14 General System Characteristics (GSCs), each rated on a scale of 0 (no influence) to 5 (strong influence).

Table 6.10: General System Characteristics (GSC) Ratings:

GSC ID	General System Characteristic	Rating	Justification
1	Data Communications	2	Web-based application; academic prototype runs on localhost.

GSC ID	General System Characteristic	Rating	Justification
2	Distributed Data Processing	1	Single MySQL instance; no distributed processing.
3	Performance	2	Not a critical concern for a demo system with low concurrent load.
4	Heavily Used Configuration	1	Runs on a single development laptop.
5	Transaction Rate	1	Low transaction rate expected in a demo environment.
6	Online Data Entry	4	Over 80% of transactions are online data entry.
7	End-User Efficiency	3	UI is designed for ease of use; advanced efficiency features limited.
8	Online Update	3	Many ILFs are updated online (plants, orders, profile).
9	Complex Processing	1	Business logic is straightforward CRUD with simple calculations.
10	Reusability	2	Code is moderately reusable; developed primarily for this project.
11	Installation Ease	3	Requires standard XAMPP and npm setup; documented in README.
12	Operational Ease	3	Easy to start/stop; manual backup processes.
13	Multiple Sites	1	Single-site deployment only.
14	Facilitate Change	2	Loosely coupled code; no formal configuration management.

Total Degree of Influence (TDI) = $2+1+2+1+1+4+3+3+1+2+3+3+1+2 = 29$

Value Adjustment Factor (VAF) = $0.65 + (29 / 100) = 0.94$

6.3.7 Counting Adjusted Function Points

Adjusted Function Points (AFP) = $170 \times 0.94 = 159.8 \approx 160$ AFP

6.3.8 Effort Estimation

For a solo developer using mature, high-productivity frameworks (Laravel 12, Next.js 16, Tailwind CSS) and leveraging pre-built patterns, a productivity rate of **53 function points per person-month** is realistic. This rate reflects the rapid prototyping capability of the chosen stack and the developer's familiarity with the tools.

Effort = $160 \text{ AFP} / 53 \text{ FP/person-month} = 3.02 \text{ person-months}$

Rounding to the nearest whole number, the total effort is **3.0 person-months**. With a single developer, this effort translates directly into a **13-week** calendar duration (1 month ≈ 4.33 weeks; $3.0 \times 4.33 \approx 13$ weeks).

6.3.9 Lines of Code Approximation

Using the adjusted function point count of 160, the total codebase size is estimated at approximately **3,000 – 3,200 lines of code** (covering PHP backend, TypeScript/React frontend, and configurations). This estimate aligns with the observed size of the existing Plantify repositories and the typical productivity of the frameworks used.

6.4 Project Schedule Chart

The 13-week project is organised into 4 sprints, executed sequentially by a single developer. The sprint breakdown matches the agile process model described in Chapter 1.

Table 6.11: Project Schedule (13-Week Sprint Plan):

Week	Sprint	Key Tasks	Deliverable / Milestone
1-3	Sprint 1	Project setup, database design, Laravel API foundation (Auth, Plants, Categories, Upload), Next.js initialisation, plant browsing UI	M1 – Backend API foundation complete
4-6	Sprint 2	Login/register pages, plant detail page, cart, checkout, orders, wishlist, profile management	M2 – Core e-commerce flow functional
7-9	Sprint 3	Reviews, Q&A, specialist dashboard, delivery role, category management, PHPUnit tests for critical controllers	M3 – User-generated content and specialist features complete
10-13	Sprint 4	RBAC middleware, admin dashboard (user/order/plant management), analytics dashboard, integration testing, security hardening, final documentation	M4 – Complete system ready for deployment

A high-level Gantt chart representation is shown below (weeks on the horizontal axis, tasks as bars). All tasks are performed solely by the student developer.

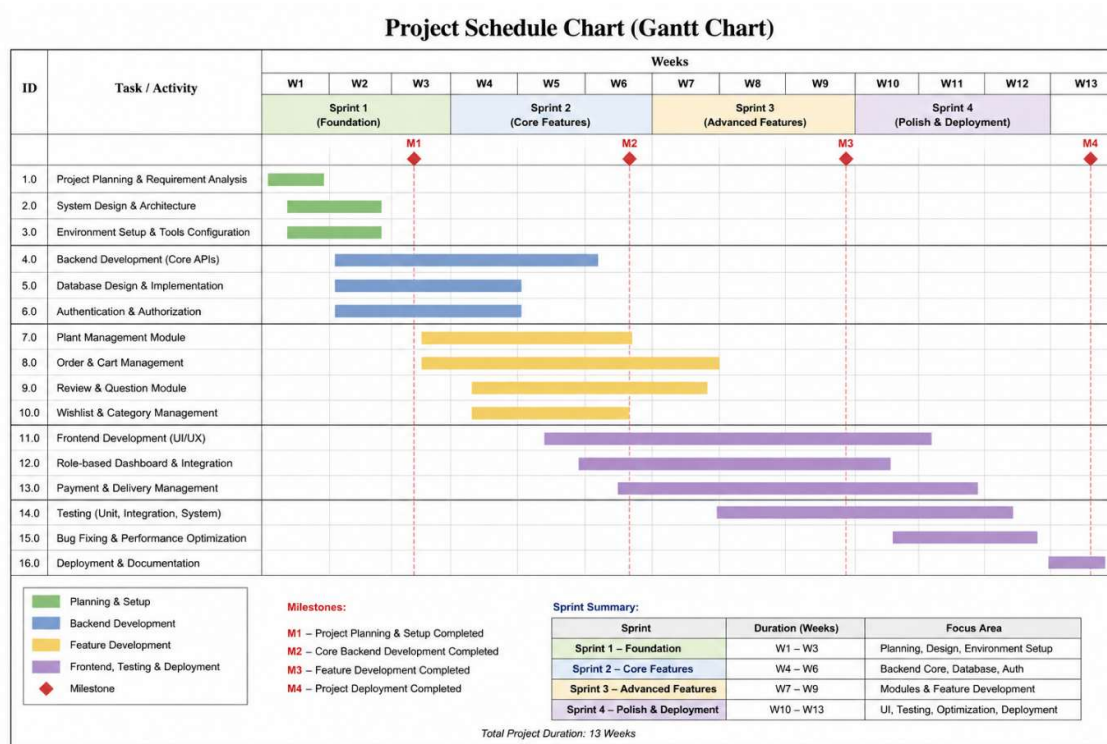


Figure 6.1: Gantt Chart – Project Schedule (13-Week Timeline)

6.5 Accounts Table

The total project cost is calculated in Bangladeshi Taka (TK). All software is free and open-source, and the development is performed by a student as part of academic coursework, resulting in a near-zero personnel cost.

6.5.1 Personnel Cost

Table 6.12: Personnel Cost:

Role	Headcount	Monthly Salary (TK)	Duration	Total Cost (TK)
Student Developer	1	0 (academic project)	13 weeks (3 months)	0
Total Personnel Cost	1			0

6.5.2 Expected Hardware Cost

Table 6.13: Expected Hardware Cost:

Item	Quantity	Total Cost (TK)	Remarks
Development Laptop (mid-range)	1	60,000 (one-time)	Cost is 0 TK if a suitable laptop is already owned.
Total Hardware Cost		0 – 60,000	

6.5.3 Expected Software Cost

All tools and frameworks are free and open-source (Visual Studio Code, Laravel, Next.js, MySQL, XAMPP, Composer, npm, Git/GitHub, Laravel Sanctum, Tailwind CSS). No licence fees are incurred.

Total Software Cost: 0 TK

6.5.4 Expected Other Cost

Table 6.14: Expected Other Cost:

Item	Cost (TK)	Remarks
Domain Name (optional)	1,800 / year	Required only if the project is publicly deployed.
Contingency	2,000	For unforeseen minor expenses.
Total Other Cost	3,800	

6.5.5 Total Project Cost Summary

Table 6.15: Total Project Cost Summary:

Category	Amount (TK)
Personnel Cost	0
Hardware Cost	0 – 60,000
Software Cost	0
Other Cost	3,800
Total Estimated Cost	3,800 – 63,800

The entire project can therefore be developed for a maximum outlay of **Tk 63,800**, and typically for only **Tk 3,800** when a development laptop is already available.

Chapter 7.

Designing

7.1 Interface Design

7.1.1 Design Principles

Table 7.1: Interface Design Principles:

Principle	Application in Plantify
Consistency	Uniform color scheme, typography, spacing, and component behavior using Tailwind CSS
Simplicity	Clean, uncluttered layouts with clear visual hierarchy
Responsiveness	Mobile-first responsive design via Tailwind breakpoints
Feedback	Visual feedback for all user actions
Accessibility	Semantic HTML, color contrast, keyboard navigation
User Control	Clear navigation, undo/cancel options, confirmation dialogs

7.1.2 Color Palette & Typography

Table 7.2: Color Palette & Typography:

Design Element	Specification
Primary Color	Green (#16a34a) — plants, growth, nature
Secondary Color	Emerald (#059669) — accents
Background	White / Gray-50 (#f9fafb)
Text Color	Gray-900 primary, Gray-600 secondary
Danger/Error	Red (#dc2626)
Font Family	Inter (system sans-serif)
Font Sizes	Headings: 24–36px; Body: 16px; Small: 14px

7.1.3 Layout Structure — Page System Design

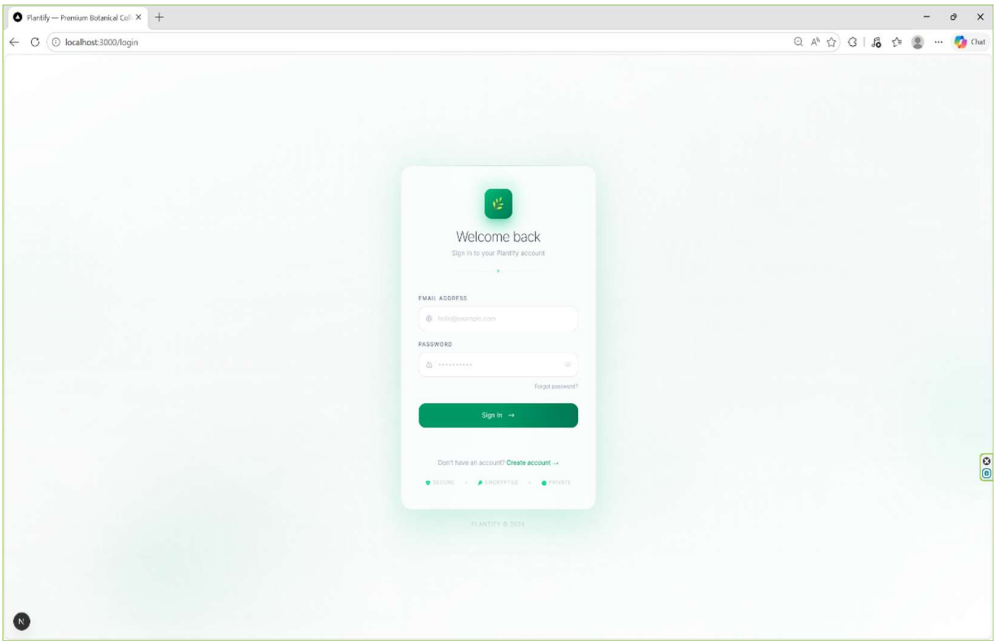


Figure 7.1: Login Page Interface Design

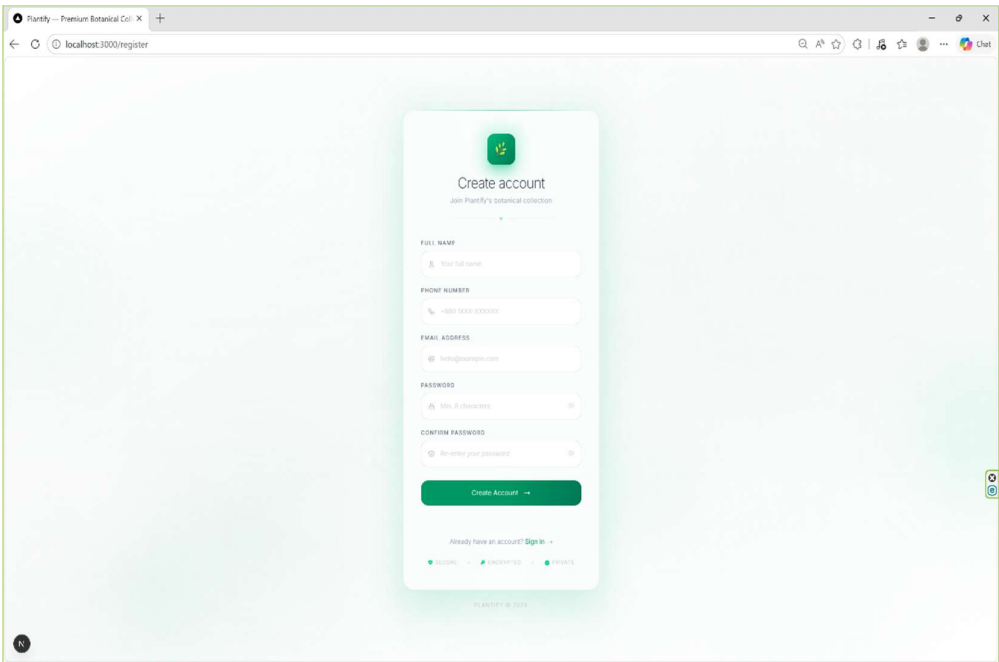


Figure 7.2: Register Page Interface Design

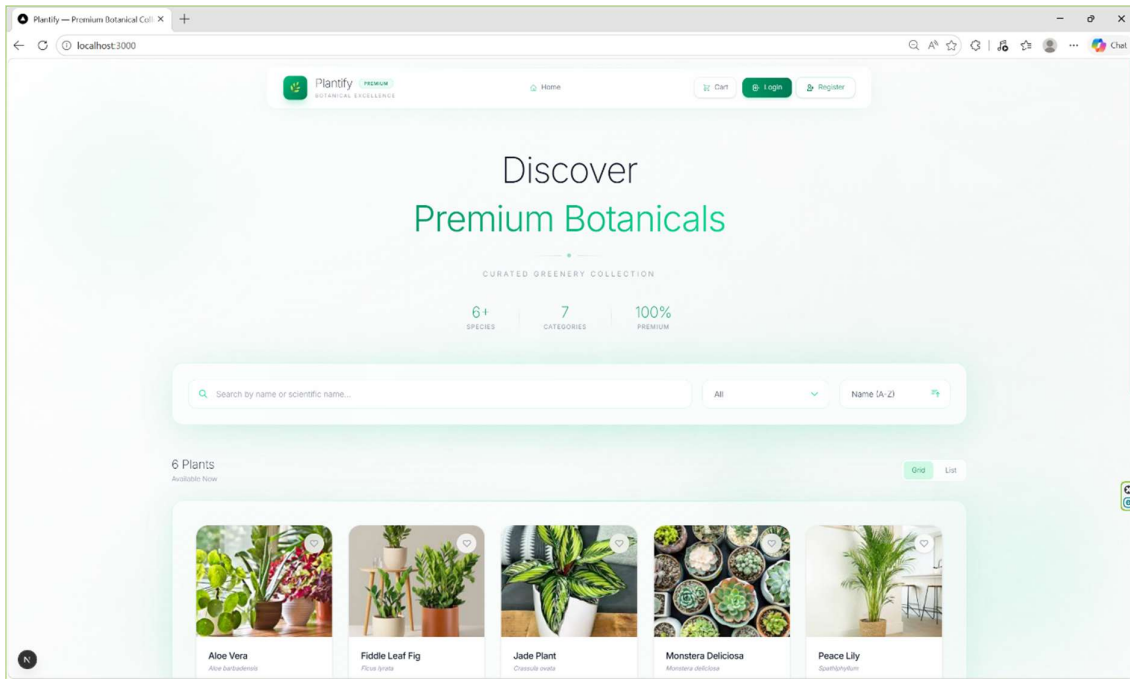


Figure 7.3: Home Page Interface Design

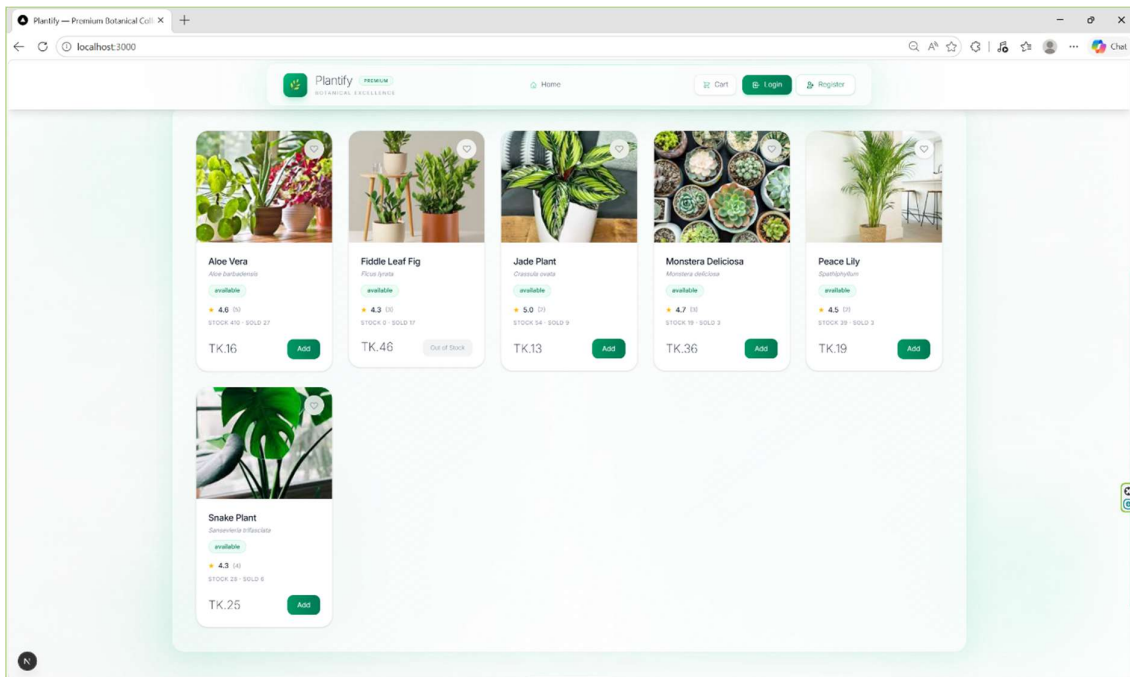


Figure 7.4: Plant Card Interface Design

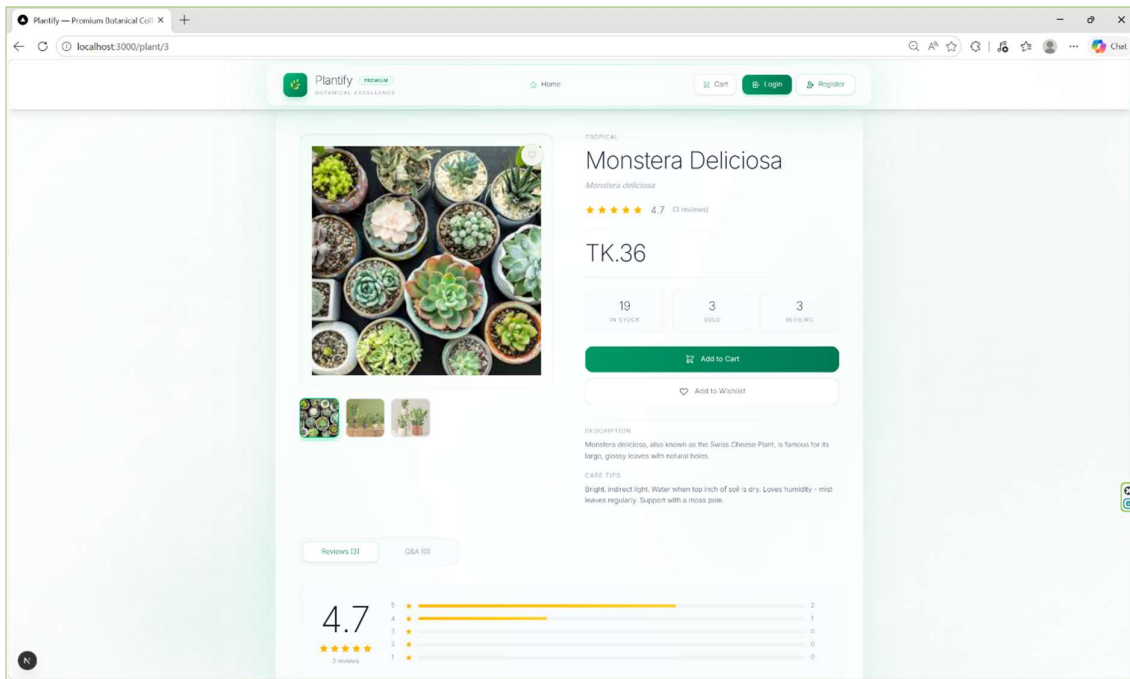


Figure 7.5: Plant Detail Page Interface Design

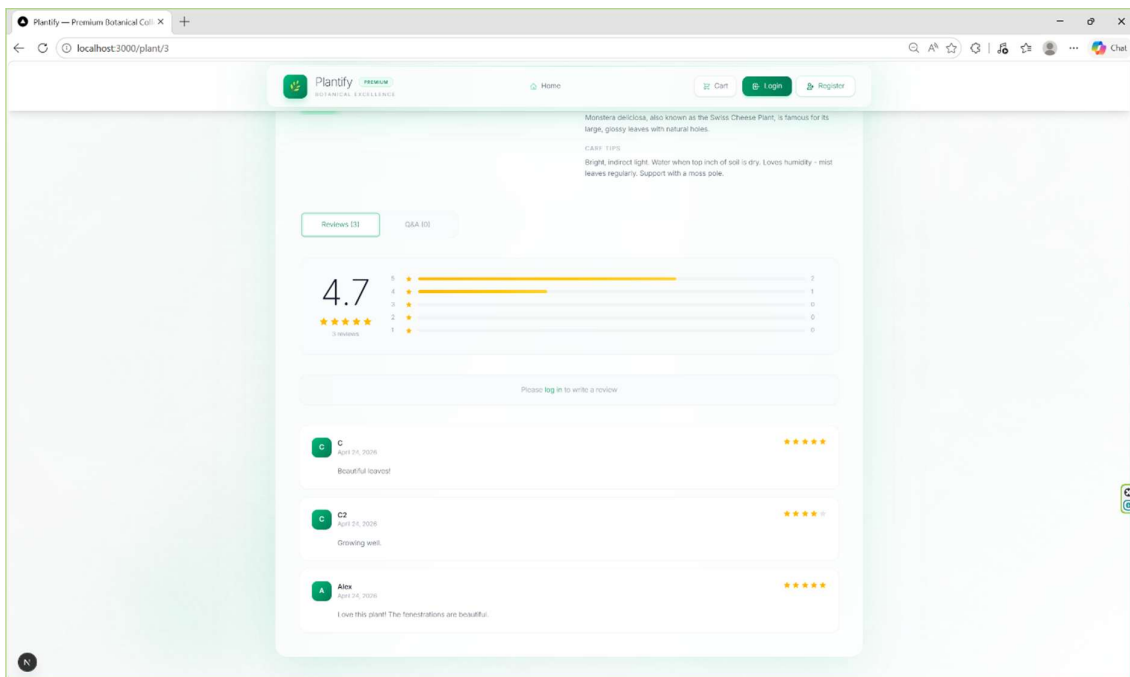


Figure 7.6: Plant Review and Q&A Interface Design

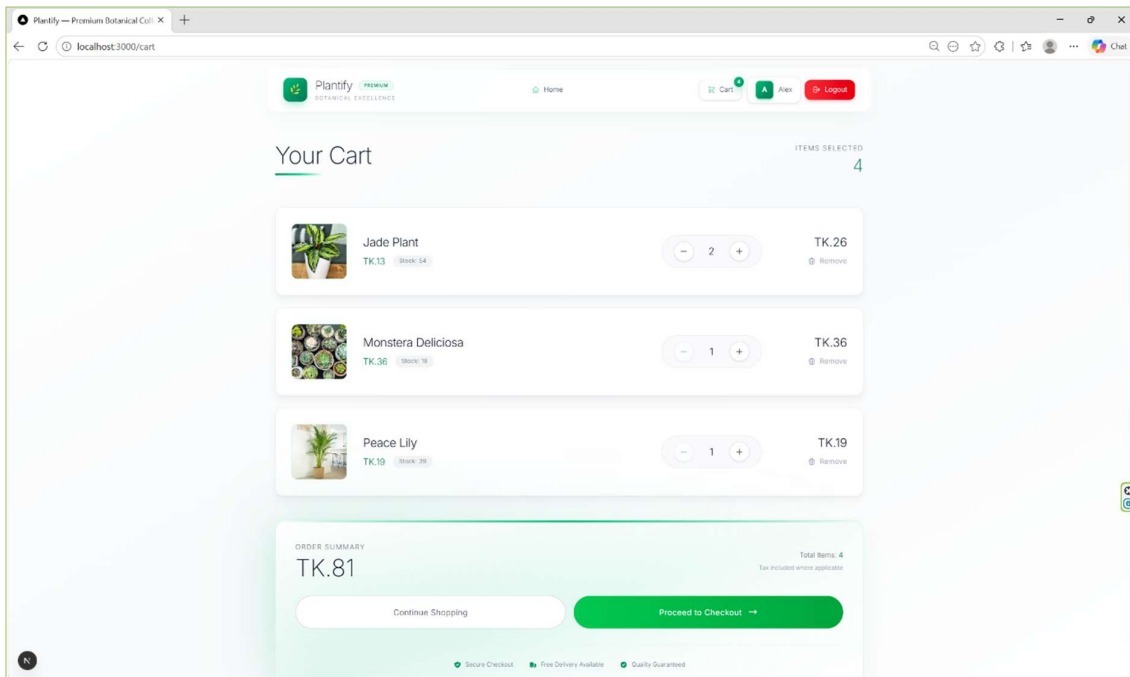


Figure 7.7: Cart Page Interface Design

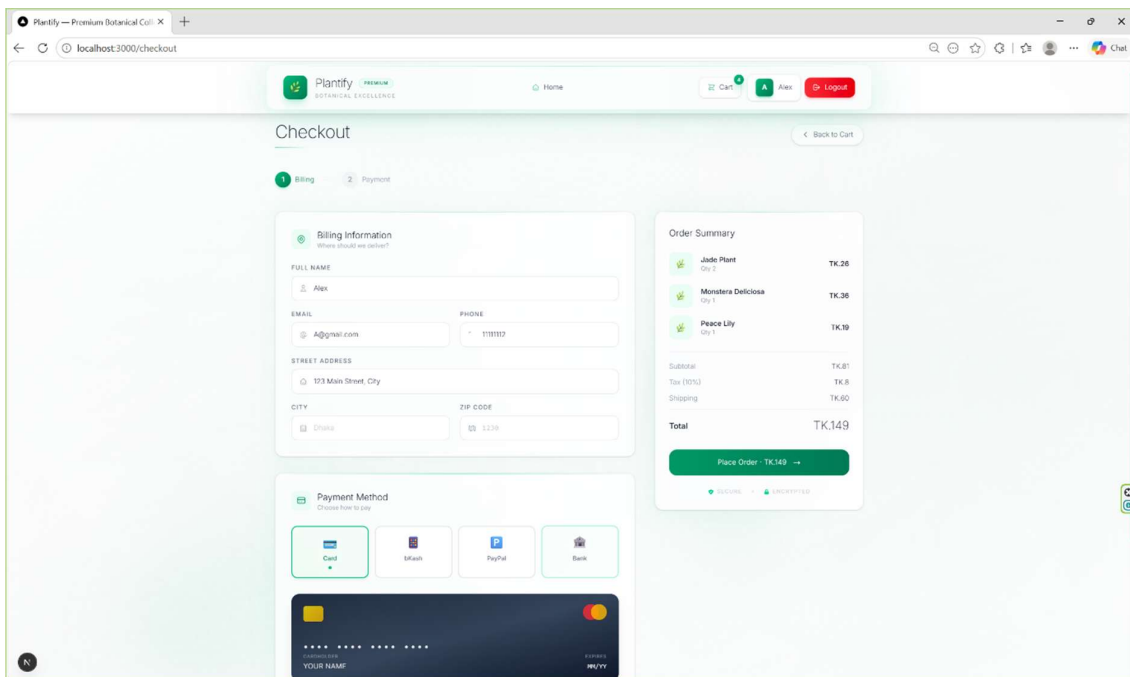


Figure 7.8: Checkout Page Interface Design

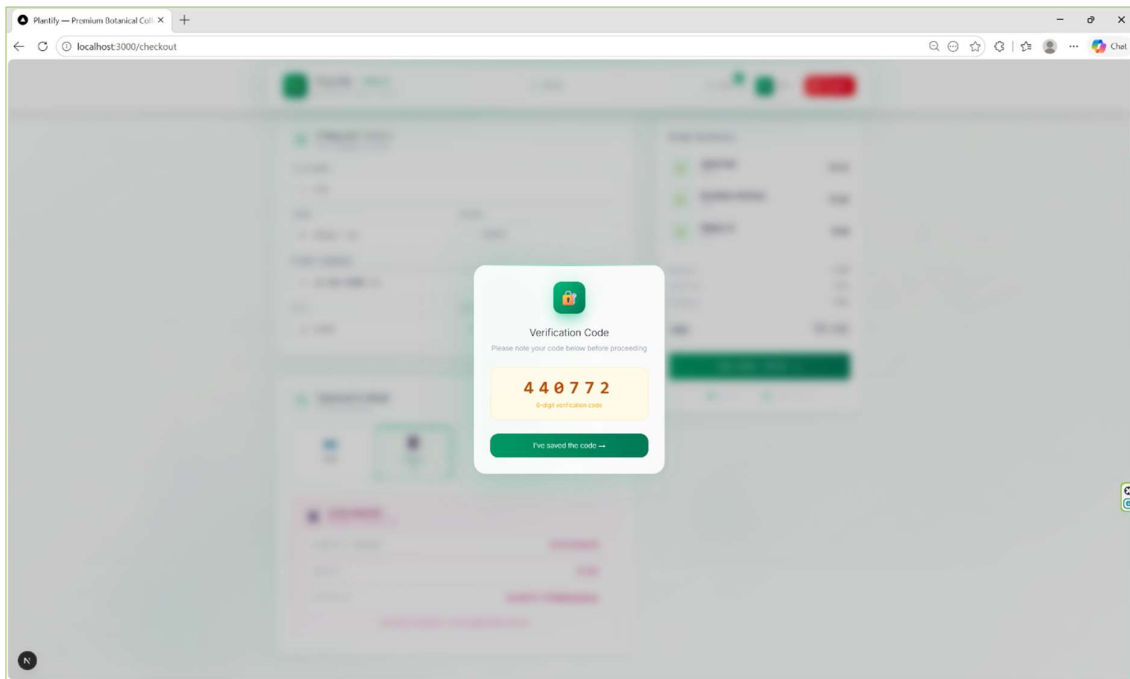


Figure 7.9: Verification Code Request Interface Design

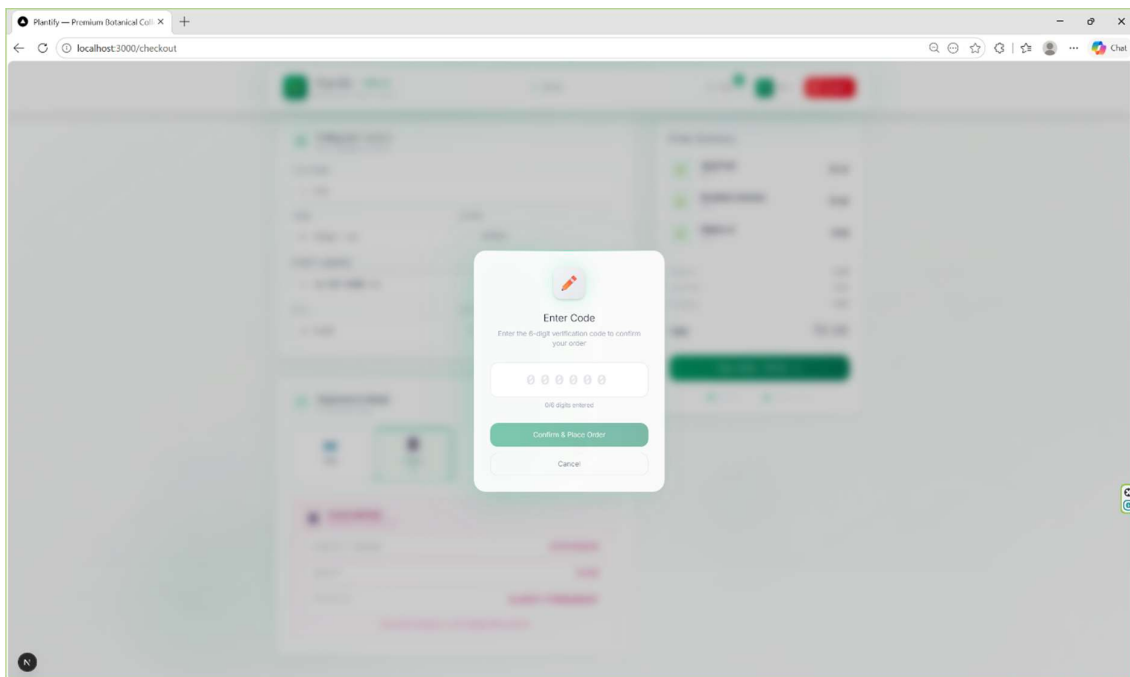


Figure 7.10: Enter Verification Code Interface Design

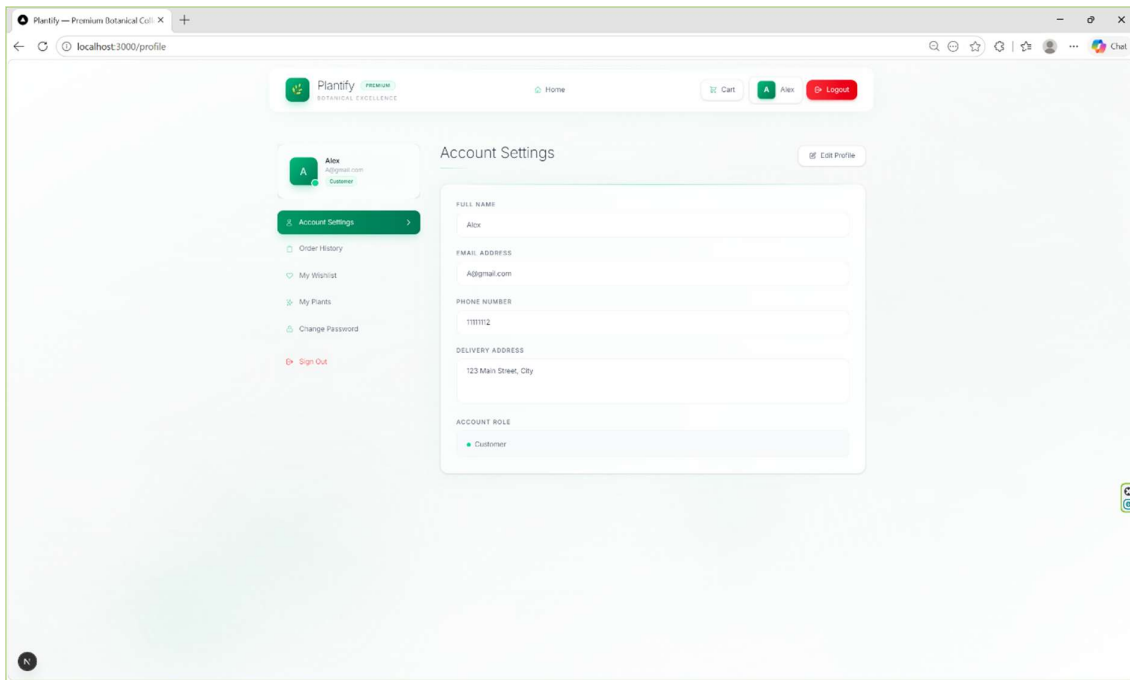


Figure 7.11: Customer Profile Page Interface Design

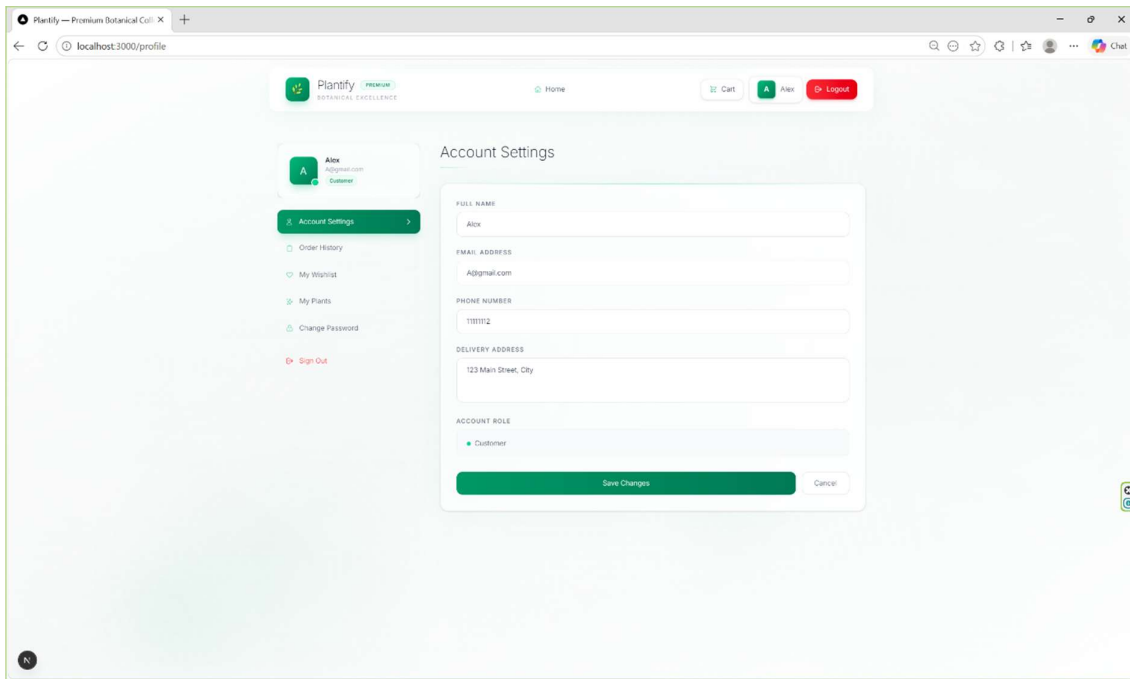


Figure 7.12: Customer Profile Edit Page Interface Design

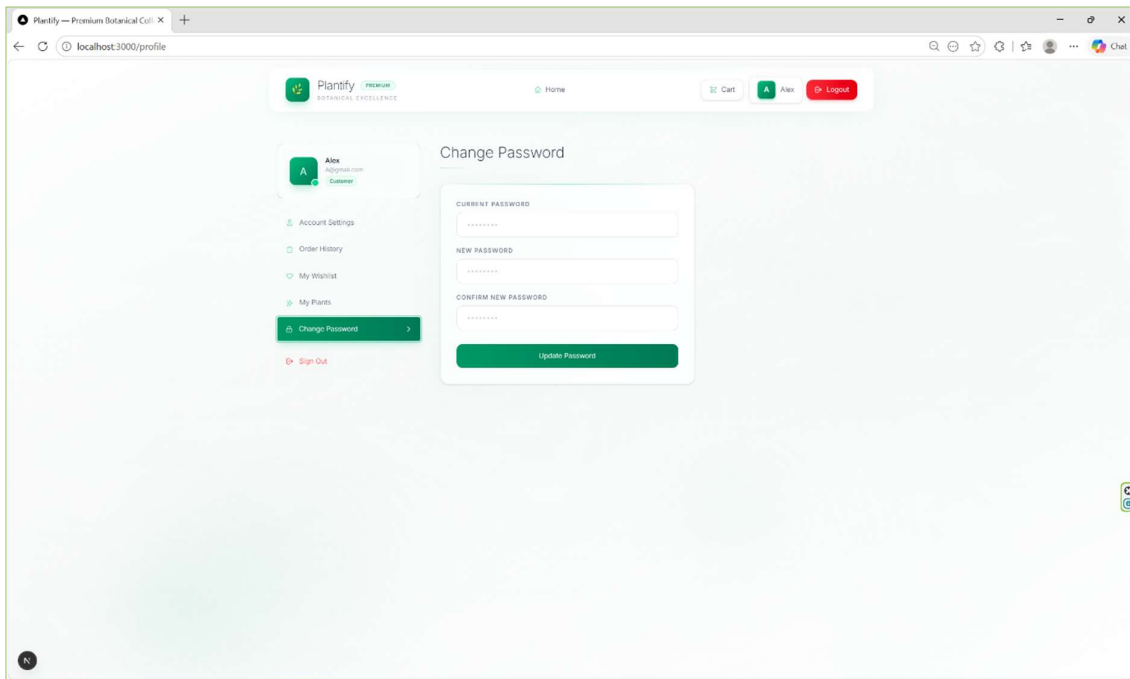


Figure 7.13: Change Password Page Interface Design

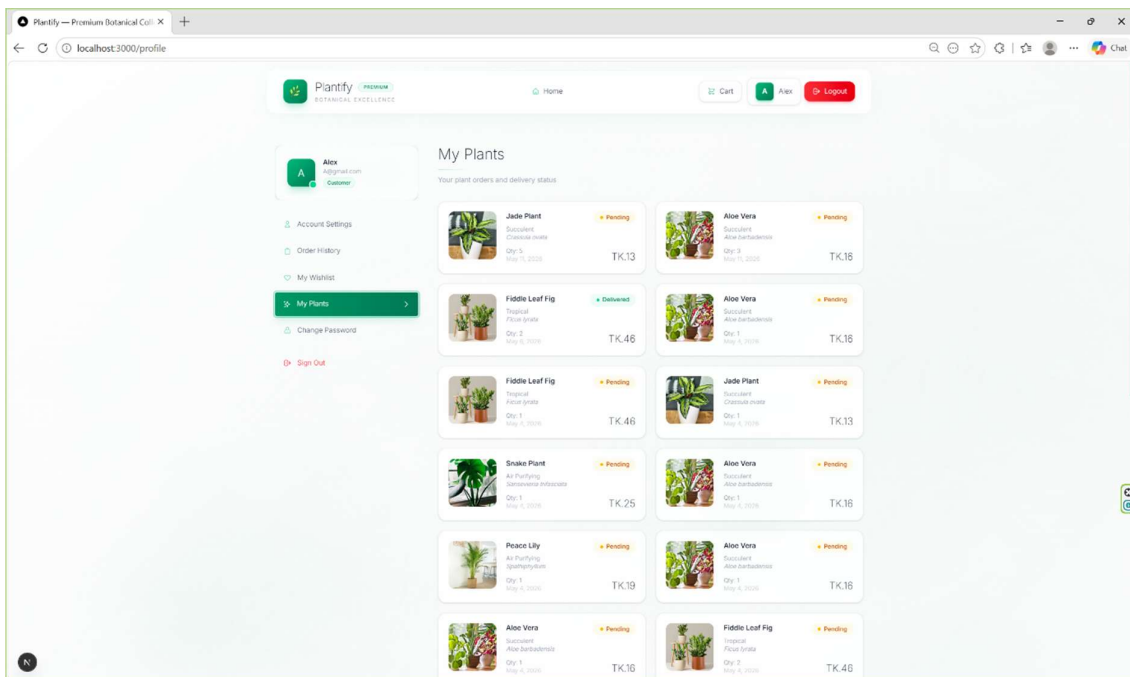


Figure 7.14: My Plants Page Interface Design

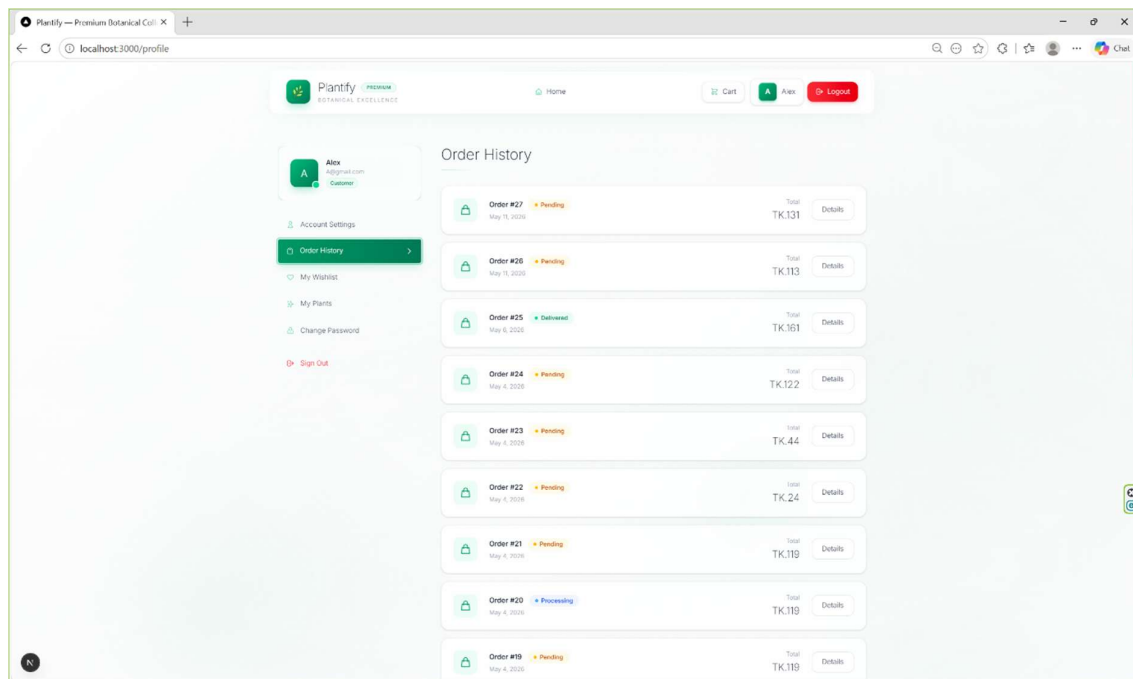


Figure 7.15: Order History Page Interface Design

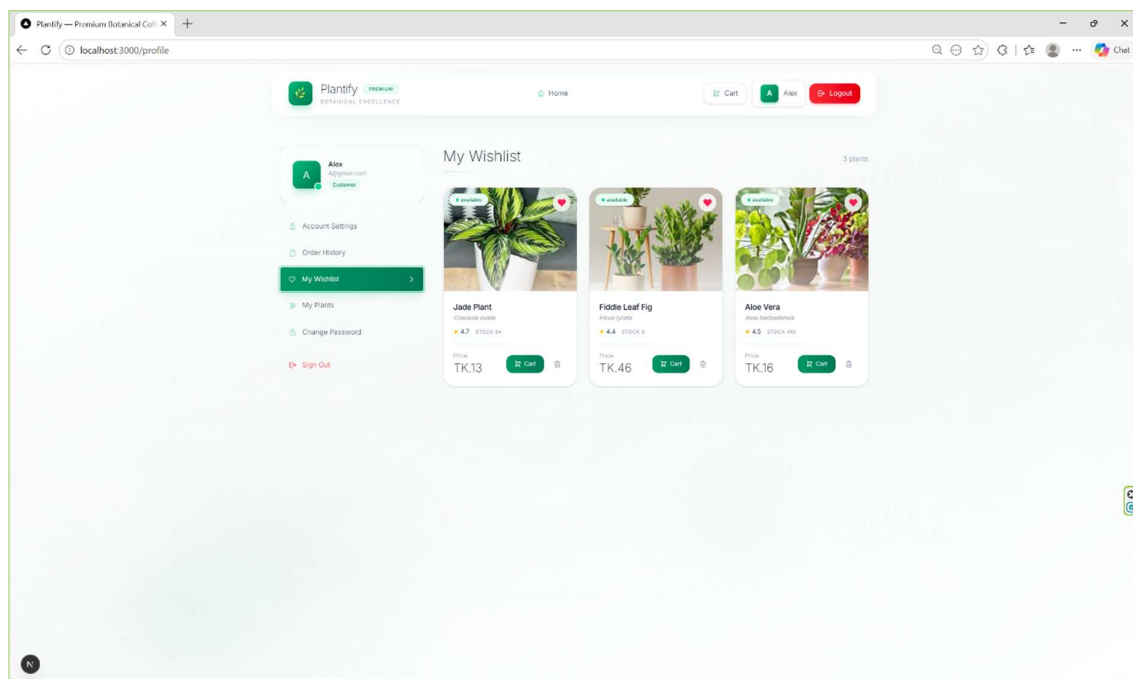


Figure 7.16: Customer Wishlist Page Interface Design